

DOCUMENTATION TECHNIQUE

Documentation Technique & Architecture du Pipeline

Table des Matières

- Football Match Manager - Technical Specification & Architecture Document 3**
 - 1. Executive Summary & Architecture Overview 3
 - 1.1 Executive Brief 3
 - 1.2 Maturity Assessment 3
 - 1.3 Technical Stack 3
 - 1.4 Architectural Constraints 3
 - 1.5 Critical Dependencies 3
 - 2. Architecture Workflows 4
 - 2.1 Project Structure Decision Flow 5
 - 2.2 Documentation Traceability Map 6
 - 3. Detailed Technical Specifications & Business Rules 7
 - 3.1 Requirements Traceability 7
 - 3.2 Security Rules 7
 - 3.3 Data Models 8
 - 4. Project Governance & Structural Gaps 8
 - 4.1 Structural Gaps 8
 - 4.2 Remediation & Workflow 8
 - 5. Technical & Domain Glossary (Terminology Reference) 8

Football Match Manager - Technical Specification & Architecture Document

1. Executive Summary & Architecture Overview

1.1 Executive Brief

The Football Match Manager project is currently in a preliminary planning phase, represented by a structural template. It aims to establish a management system for football matches, though the core value proposition and data patterns are not yet defined. The project is centered on selecting one of three architectural patterns: a monolithic single project, a client-server web application, or a mobile-backend split.

1.2 Maturity Assessment

The project is in a critical state of under-specification. While the documentation skeleton is complete, the absence of data models, API contracts, and security definitions constitutes high-severity structural gaps. Consequently, the project is currently in REFINEMENT status, as it lacks the technical substance required for implementation.

1.3 Technical Stack

- **Languages/Frameworks:** TBD (Placeholders mentioned: Python 3.11, Swift 5.9, Rust 1.75)
- **Primary Dependencies:** TBD (Placeholders mentioned: FastAPI, UIKit, LLVM)
- **Storage:** TBD (Placeholders mentioned: PostgreSQL, CoreData)
- **Testing:** TBD (Placeholders mentioned: pytest, XCTest, cargo test)
- **Target Platform:** TBD (Placeholders mentioned: Linux server, iOS 15+, WASM)

1.4 Architectural Constraints

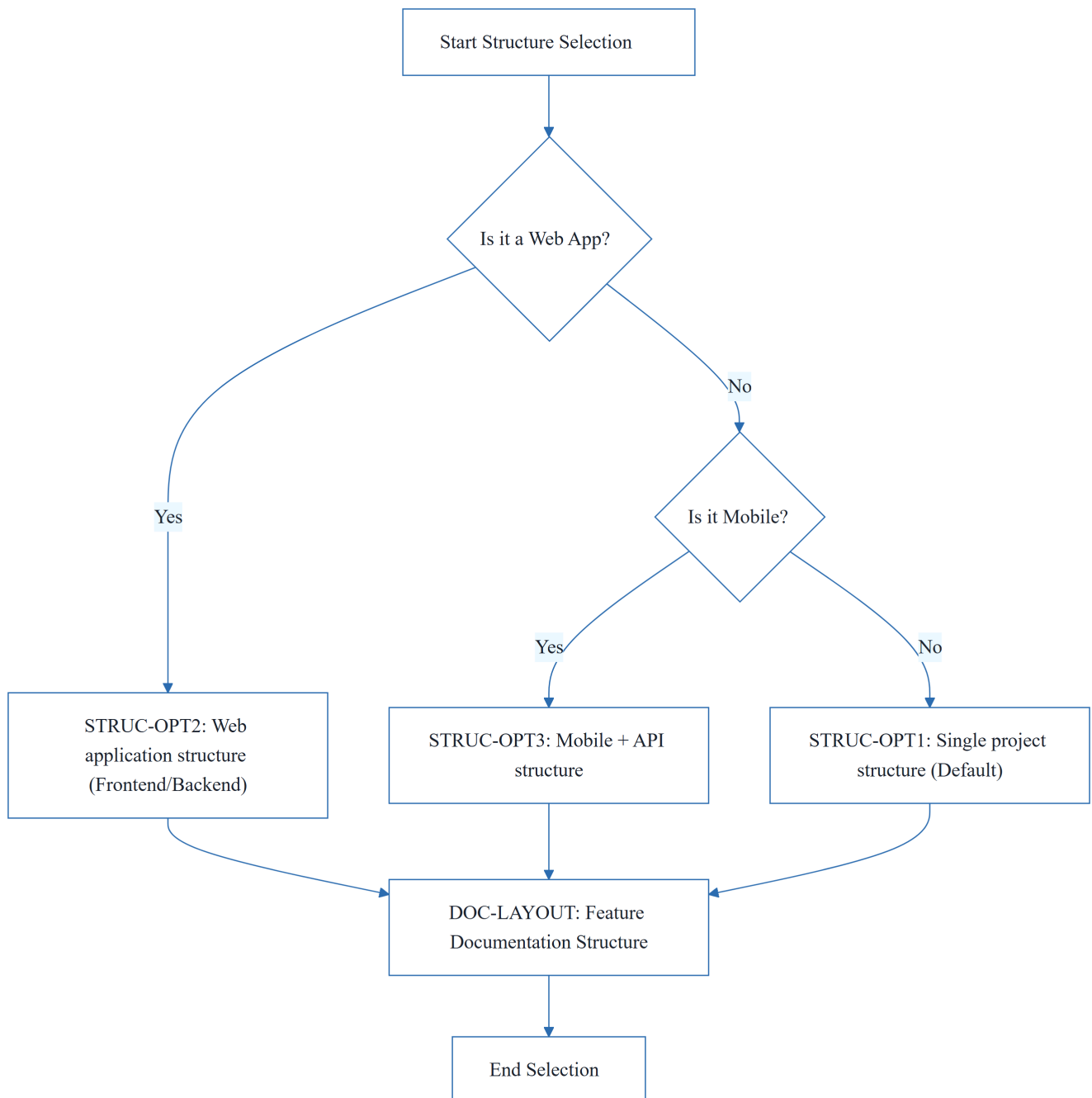
- **Structural Adherence:** Must adhere to one of three predefined project structures: Single Project (src/models/services/cli/lib), Web Application (backend/frontend), or Mobile + API.
- **Documentation Standard:** Mandatory documentation hierarchy following the specs/[###-feature]/ pattern including plan.md, research.md, data-model.md, quickstart.md, contracts/, and tasks.md.

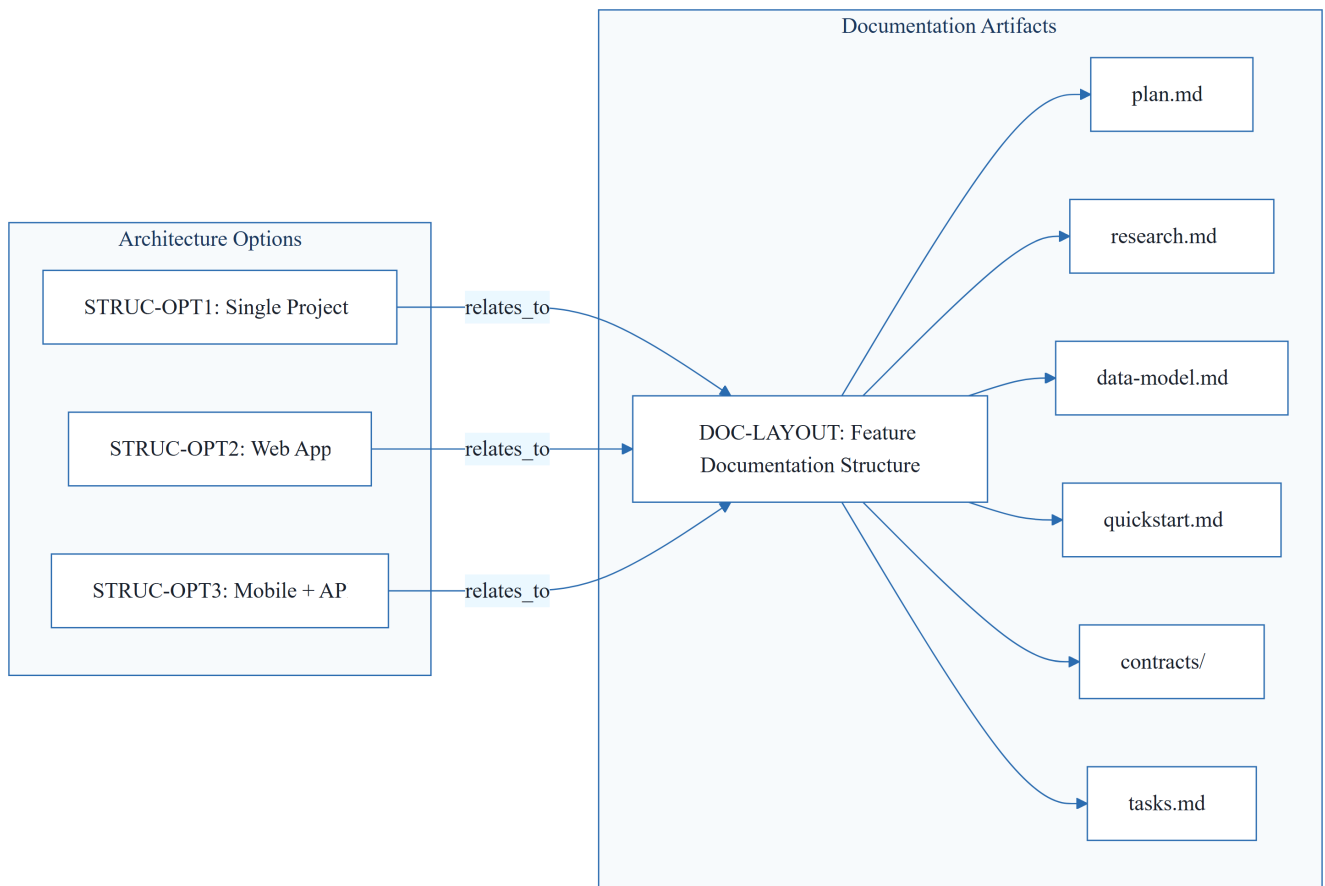
1.5 Critical Dependencies

- **Structure Selection:** Selection of a specific architectural structure (STRUC-OPT1, STRUC-OPT2, or STRUC-OPT3) is a prerequisite for codebase initialization.

- **Documentation Alignment:** Strict alignment between the chosen structure and the feature documentation layout (DOC-LAYOUT).

2. Architecture Workflows

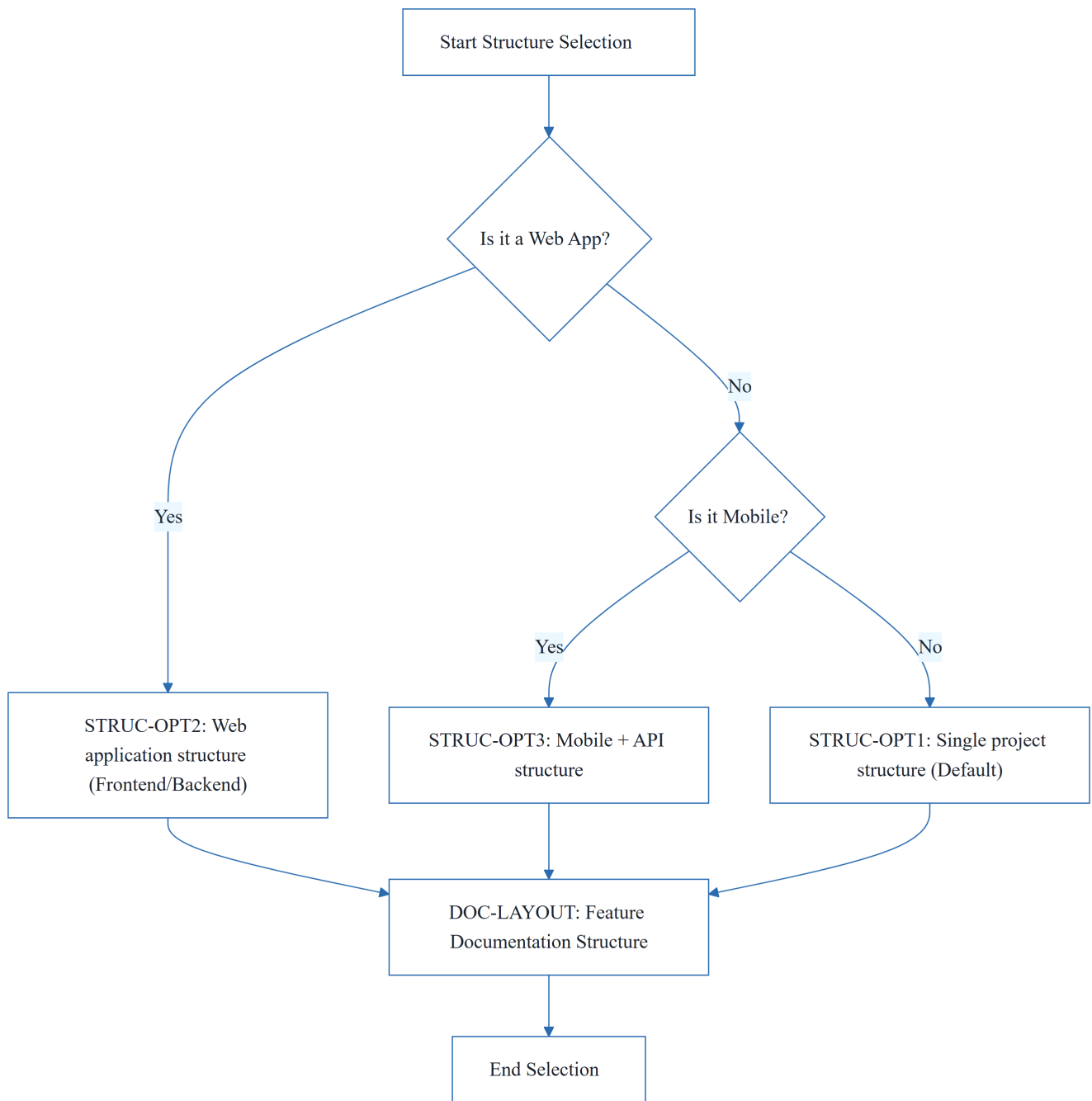




& Visual Diagrams

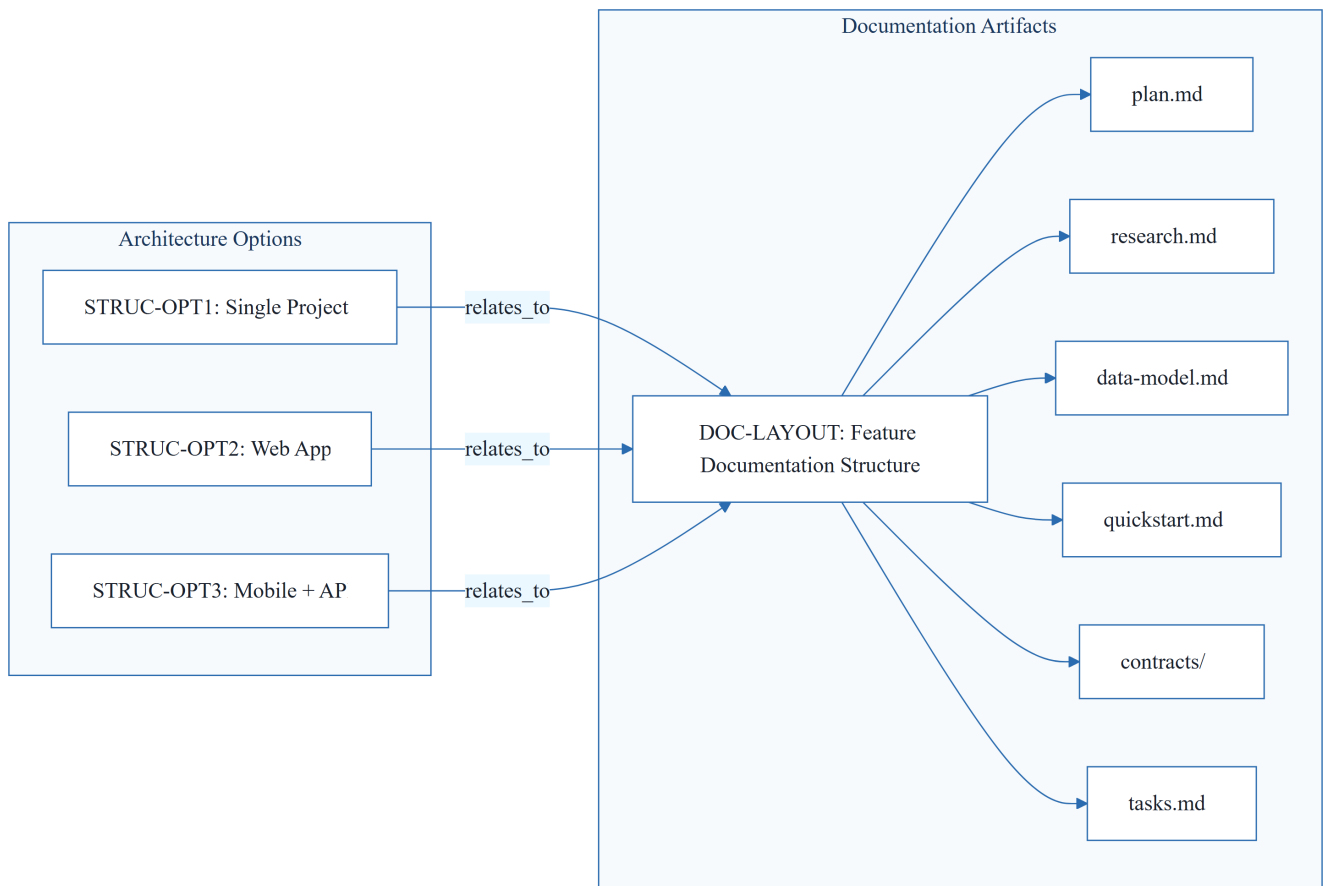
2.1 Project Structure Decision Flow

This flowchart describes the decision logic for selecting the appropriate project source code structure based on project requirements.



2.2 Documentation Traceability Map

Mapping of the documentation hierarchy and its relationship to the architectural choices.



3. Detailed Technical Specifications & Business Rules

3.1 Requirements Traceability

Identifier	Requirement / Choice Description	Source Section
DOC-LAYOUT	Feature documentation structure consisting of plan, research, data-model, quickstart, contracts, and tasks.	Documentation (this feature)
STRUC-OPT1	Single project structure with src (models, services, cli, lib) and tests (contract, integration, unit).	[REMOVE IF UNUSED] Option 1: Single project (DEFAULT)
STRUC-OPT2	Web application structure split into backend (models, services, api) and frontend (components, pages, services).	[REMOVE IF UNUSED] Option 2: Web application
STRUC-OPT3	Mobile + API structure split into api and platform-specific mobile folders (ios/android).	[REMOVE IF UNUSED] Option 3: Mobile + API

3.2 Security Rules

- **Status:** NOT DEFINED.
- **Gap:** Security and Identity mechanisms are currently missing from the specification.

3.3 Data Models

- **Status:** NOT DEFINED.
- **Gap:** Entities and database schemas for the Football Match Manager are currently missing.

4. Project Governance & Structural Gaps

4.1 Structural Gaps

Missing Section	Priority	Remediation Advice
Data Models & Schemas	HIGH	Define the entities and database schema for the Football Match Manager.
API Contracts & Flow	HIGH	Define the REST/GraphQL endpoints and communication flows between components.
Security & Identity	MEDIUM	Specify authentication, authorization, and data protection mechanisms.
Open Questions & Uncertainties	LOW	List any remaining technical doubts after the research phase.

4.2 Remediation & Workflow

The project must transition from the current "Template" state to a "Defined" state by executing the following workflow:

1. **Phase 0 (Research):** Define the core value proposition and technical approach.
2. **Phase 1 (Design):** Select one of the STRUC-OPT patterns and define the `data-model.md` and `contracts/`.
3. **Phase 2 (Execution):** Generate `tasks.md` based on the finalized technical specifications.

5. Technical & Domain Glossary (Terminology Reference)

Term	Category	Context Anchor	Project Definition
Branch	TECHNICAL_STACK	Implementation Plan: Football Match Manager	The version control pointer designated as 001-football-match-manager.
Date	TECHNICAL_STACK	Implementation Plan: Football Match Manager	The temporal marker set to 2026-07-30.

Term	Category	Context Anchor	Project Definition
Spec	TECHNICAL_STACK	Implementation Plan: Football Match Manager	The technical requirement document located at spec.md.
Python 3.11	TECHNICAL_STACK	Technical Context	An illustrative language version used as a placeholder for the execution environment.
Primary Dependencies	TECHNICAL_STACK	Technical Context	External libraries or frameworks such as FastAPI or UIKit serving as foundational building blocks.
CoreData	TECHNICAL_STACK	Technical Context	An example persistent framework for local object management.
Storage	TECHNICAL_STACK	Technical Context	The persistence layer mechanism, such as PostgreSQL or local files.
Testing	TECHNICAL_STACK	Technical Context	The validation suite utilizing tools like pytest or XCTest.
Target Platform	TECHNICAL_STACK	Technical Context	The intended deployment environment such as Linux server or WASM.
Project Type	TECHNICAL_STACK	Technical Context	The architectural classification, for instance, a web-service or mobile-app.
Performance Goals	TECHNICAL_STACK	Technical Context	Quantifiable efficiency targets such as 60 fps or 1000 req/s.
Constraints	TECHNICAL_STACK	Technical Context	Strict operational limits including <200ms p95 or <100MB memory.
LOC	TECHNICAL_STACK	Technical Context	A metric measuring the total volume of written code, exemplified by 1M units.
GATE	TECHNICAL_STACK	Constitution Check	A mandatory validation checkpoint that must be cleared before proceeding to research or design phases.
LLVM	TECHNICAL_STACK	Technical Context	A compiler infrastructure example listed as a potential dependency.
WASM	TECHNICAL_STACK	Technical Context	A binary instruction format used as an example target execution environment.

Term	Category	Context Anchor	Project Definition
iOS	TECHNICAL_STACK	STRUC-OPT3	The Apple mobile operating system environment requiring specific directory structures for UI flows.
UI	TECHNICAL_STACK	STRUC-OPT3	The visual interface layer containing platform-specific flow modules.
API	TECHNICAL_STACK	STRUC-OPT2	The interface layer managing communication between the backend and frontend components.
Structure Decision	TECHNICAL_STACK	STRUC-OPT3	The final selection of the repository layout among the provided options.